LAMBDAS, CURRYING, AND ENVIRONMENTS Meta

COMPUTER SCIENCE MENTORS 61A

September 14, 2026 - September 18, 2026

Example Timeline

- **Each problem's "Suggested Time" is not assigned with the 1 hour time for sections in mind.** It is more of a note of how long each individual problem may take. That is, if you added all the "Suggested Times" in the coding section, it could very well be greater than 1 hour (especially in more complicated worksheets in the future).

  The times in brackets are intended as a suggested outline; choose or skip problems as needed to fit your section.

- Intros, Ice Breakers, & Logistics [10 min]

  Introduce yourself to your students. They will probably be your students for the rest of the semester. Start off with an icebreaker of your choice! Also, explain logistics like doing CSM for a unit and how many absences are allowed and etc.

- How to tackle terrified students after the midterm [5 min]

  Your students are also a bit terrified by the midterm which happened this past Monday so use any of the techniques which you have discussed in family meeting to alleviate them.

  Try to start the session off by acknowledging the midterm and the feelings around it (even ask them how they felt about it) and make them and all of their feelings and questions feel included. Tell them that a lot of people feel in the same way as they do and be realistic.

  Ask them what is one thing that felt hard to them during the midterm: reading questions, tracing code, writing functions, or something else and take a note of that so that when you prep for future sessions you can modify your teaching style to fit your students better.

- Expressions and truth values warm-up [8 min]
- Lambdas and converting function forms [10 min]
- Currying and zero-argument functions [12 min]
- Environment diagrams with functions and lambdas [20 min]
- Function implementation [10 min]

Created by Cadence Loh and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1 Expressions and Truth Values

1.
```
def quick_maths(man, free):
    print(man)
    if man % 2 == 0 and free:
        return quick_maths(man + 2, not free)
    else:
        man -= 1
        if man == 3:
            print(man)
        else:
            return man
big_shaq = 2
```

Expression	Interactive Output
`print(quick_maths(big_shaq, True))`	

Expression	Interactive Output

`print(quick_maths(big_shaq, True))`	`2`
	`4`
	`3`
	`None`

Use the condition `man \% 2 == 0 and free` to review short-circuiting and the distinction between `True`/`False` values and the value an **`and`** expression returns.

2 Lambda Expressions and Function Forms

1. What do lambda expressions do?

   Lambda expressions create functions in the environment. When you evaluate one, you get out a function (feel free to call them a lambda function) as a value.

2. Rewrite each function in the requested equivalent form. Preserve the function's parameters and behavior; only change whether it is written with **`def`** or **`lambda`**. For your reference, a **`lambda`** assignment assigns a lambda function to a variable.

   (a) Rewrite the following **`def`** as a lambda assignment.

   ```
   def square(x):
       return x * x
   ```

   ```
   square = lambda x: x * x
   ```

   (b) Rewrite the following lambda assignment using **`def`**.

   ```
   subtract = lambda x, y: x - y
   ```

   ```
   def subtract(x, y):
       return x - y
   ```

   (c) Rewrite this zero-argument function as a lambda assignment.

   ```
   def greeting():
       return 'Hello, world!'
   ```

   ```
   greeting = lambda: 'Hello, world!'
   ```

3 Currying and Zero-Argument Functions

1. Fill in the blanks (*without using any numbers in the first blank*) such that the entire expression evaluates to 9.

   ```
   (lambda x: lambda y: lambda: y(x))(3)(lambda z: z*z)()
   ```

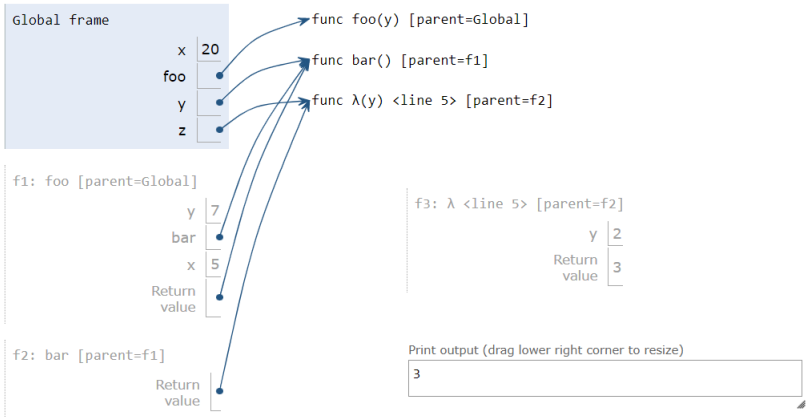
   This question combines currying, lambdas, and a zero-argument call. Have students identify the value returned at each pair of parentheses.

4 Environment Diagrams with Lambdas

1. Give the environment diagram and console output that result from running the following code.

```
x = 20
def foo(y):
    x = 5
    def bar():
        return lambda y: x - y
    return bar

y = foo(7)
z = y()
print(z(2))
```



https://tinyurl.com/yxfcvxxa

Before tracing, ask: "What frame does each function have as its parent?" Emphasize that calling `y()` creates a frame even though it has no parameters.

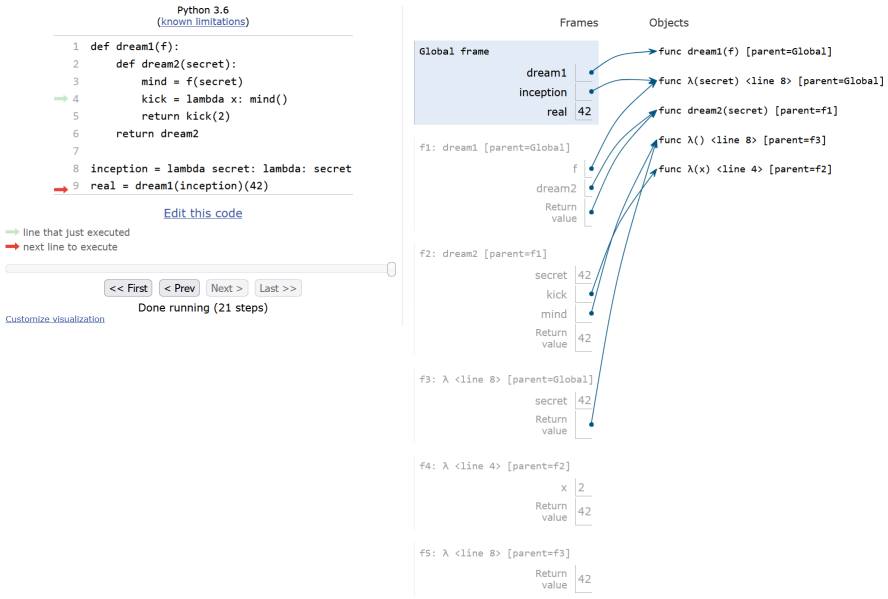
2. Draw the environment diagram that results from running the code below.

It may be helpful to give students an overview on the rules of drawing environment diagrams, such as when to open new frames, how to format the frames, what parent frames are, when to draw pointers to objects, frame hierarchy, lambdas, and any tips/tricks you may think of!

```
def dream1(f):
    def dream2(secret):
        mind = f(secret)
        kick = lambda x: mind()
        return kick(2)
    return dream2

inception = lambda secret: lambda: secret
real = dream1(inception)(42)
```

Output: 42



https://imgur.com/a/ZKwZzdy

This is a stretch problem. Track function values carefully: a call may return a function, and a lambda's parent is the frame in which it is created.

5 Basic Function Implementation

1. Write a higher-order function that passes the following doctests.

   Challenge: Write the function body in one line.

```
def mystery(f, x):
    """
    >>> from operator import add, mul
    >>> a = mystery(add, 3)
    >>> a(4) # add(3, 4)
    7
    >>> a(12)
    15
    >>> b = mystery(mul, 5)
    >>> b(7) # mul(5, 7)
    35
    >>> b(1)
    5
    >>> c = mystery(lambda x, y: x * x + y, 4)
    >>> c(5)
    21
    >>> c(7)
    23
    """


    def helper(y):
        return f(x, y)
    return helper
```

   Challenge solution:

```
    return lambda y : f(x, y)
```

   Encourage students to write the nested **def** solution first, then compare it with the one-line lambda challenge solution.

2. Implement `make_skipper`, which takes in a number `n` and outputs a function. When this function takes in a number `x`, it prints out all the numbers between `0` and `x`, skipping every `n`th number.

```
def make_skipper(n):
    """
    >>> a = make_skipper(2)
    >>> a(5)
    1
    3
    5
    """


    def skipper(x):
        for i in range(x + 1):
            if i % n != 0:
                print(i)
    return skipper
```